Dynamique du λ-calcul

Principe de réduction

$$\begin{array}{lrll} \lambda\text{-terme} & t, u ::= & x & \text{(variable)} \\ & & \lambda x.t & \text{(abstraction)} \\ & & t\ u & \text{(application)} \end{array}$$

- redex (*reducible expression*) : fonction appliquée à un argument

$$(\lambda x.t)\ u$$

- β-réduction : substitution de la variable avec l'argument

$$(\lambda x.t)\ u \to_\beta t\{u/x\}$$

- Possible n'importe où dans le terme

Exemple

$$(\lambda x.x)\ ((\lambda x.x)\ \lambda z.(\lambda x.x)\ z)$$

Exemple

$\mathrm{id} = \lambda x.x$

$$\mathrm{id}\ (\mathrm{id}\ \lambda z.\mathrm{id}\ z)$$

Exemple

$\mathrm{id} = \lambda x.x$

$$\mathrm{id}\ (\mathrm{id}\ \lambda z.\mathrm{id}\ z)$$

Trois redex :

$$\underline{\mathrm{id}\ (\mathrm{id}\ \lambda z.\mathrm{id}\ z)} \qquad \mathrm{id}\ (\underline{\mathrm{id}\ \lambda z.\mathrm{id}\ z}) \qquad \mathrm{id}\ (\mathrm{id}\ \lambda z.\underline{\mathrm{id}\ z})$$

Exemple

$\mathrm{id} = \lambda x.x$

$$\mathrm{id}\ (\mathrm{id}\ \lambda z.\mathrm{id}\ z)$$

Trois redex :

$$\underline{\mathrm{id}\ (\mathrm{id}\ \lambda z.\mathrm{id}\ z)} \qquad \mathrm{id}\ (\underline{\mathrm{id}\ \lambda z.\mathrm{id}\ z}) \qquad \mathrm{id}\ (\mathrm{id}\ \lambda z.\underline{\mathrm{id}\ z})$$

$$\mathrm{id}\ (\mathrm{id}\ (\lambda z.\underline{\mathrm{id}\ z}))$$

Exemple

$\mathsf{id} = \lambda x.x$

$$\mathsf{id}\ (\mathsf{id}\ \lambda z.\mathsf{id}\ z)$$

Trois redex :

$$\underline{\mathsf{id}\ (\mathsf{id}\ \lambda z.\mathsf{id}\ z)} \qquad \mathsf{id}\ (\underline{\mathsf{id}\ \lambda z.\mathsf{id}\ z}) \qquad \mathsf{id}\ (\mathsf{id}\ \lambda z.\underline{\mathsf{id}\ z})$$

$$\mathsf{id}\ (\mathsf{id}\ (\lambda z.\underline{\mathsf{id}\ z})) \to_\beta \mathsf{id}\ (\mathsf{id}\ \lambda z.z)$$

Exemple

$\mathsf{id} = \lambda x.x$

$$\mathsf{id}\ (\mathsf{id}\ \lambda z.\mathsf{id}\ z)$$

Trois redex :

$$\underline{\mathsf{id}\ (\mathsf{id}\ \lambda z.\mathsf{id}\ z)} \qquad \mathsf{id}\ (\underline{\mathsf{id}\ \lambda z.\mathsf{id}\ z}) \qquad \mathsf{id}\ (\mathsf{id}\ \lambda z.\underline{\mathsf{id}\ z})$$

$$\mathsf{id}\ (\mathsf{id}\ (\lambda z.\underline{\mathsf{id}\ z})) \to_\beta \underline{\mathsf{id}\ (\mathsf{id}\ \lambda z.z)}$$

Exemple

$\mathsf{id} = \lambda x.x$

$$\mathsf{id}\ (\mathsf{id}\ \lambda z.\mathsf{id}\ z)$$

Trois redex :

$$\underline{\mathsf{id}\ (\mathsf{id}\ \lambda z.\mathsf{id}\ z)} \qquad \mathsf{id}\ (\underline{\mathsf{id}\ \lambda z.\mathsf{id}\ z}) \qquad \mathsf{id}\ (\mathsf{id}\ \lambda z.\underline{\mathsf{id}\ z})$$

$$\begin{aligned}\mathsf{id}\ (\mathsf{id}\ (\lambda z.\underline{\mathsf{id}\ z})) &\to_\beta \underline{\mathsf{id}\ (\mathsf{id}\ \lambda z.z)} \\ &\to_\beta \mathsf{id}\ \lambda z.z\end{aligned}$$

Exemple

$\mathsf{id} = \lambda x.x$

$$\mathsf{id}\ (\mathsf{id}\ \lambda z.\mathsf{id}\ z)$$

Trois redex :

$$\underline{\mathsf{id}\ (\mathsf{id}\ \lambda z.\mathsf{id}\ z)} \qquad \mathsf{id}\ (\underline{\mathsf{id}\ \lambda z.\mathsf{id}\ z}) \qquad \mathsf{id}\ (\mathsf{id}\ \lambda z.\underline{\mathsf{id}\ z})$$

$$\begin{aligned}\mathsf{id}\ (\mathsf{id}\ (\lambda z.\underline{\mathsf{id}\ z})) &\to_\beta \underline{\mathsf{id}\ (\mathsf{id}\ \lambda z.z)} \\ &\to_\beta \underline{\mathsf{id}\ \lambda z.z} \\ &\to_\beta \lambda z.z\end{aligned}$$

Exemple

$\mathsf{id} = \lambda x.x$

$$\mathsf{id}\ (\mathsf{id}\ \lambda z.\mathsf{id}\ z)$$

Trois redex :

$$\underline{\mathsf{id}\ (\mathsf{id}\ \lambda z.\mathsf{id}\ z)} \qquad \mathsf{id}\ (\underline{\mathsf{id}\ \lambda z.\mathsf{id}\ z}) \qquad \mathsf{id}\ (\mathsf{id}\ \lambda z.\underline{\mathsf{id}\ z})$$

$$\begin{aligned}\mathsf{id}\ (\mathsf{id}\ (\lambda z.\underline{\mathsf{id}\ z})) &\to_\beta \underline{\mathsf{id}\ (\mathsf{id}\ \lambda z.z)} \\ &\to_\beta \underline{\mathsf{id}\ \lambda z.z} \\ &\to_\beta \lambda z.z\end{aligned}$$

Exercice : Écrire une autre séquence de réductions possible

β-réduction

- Combien de redex dans $(\lambda x.(\lambda y.y)\,(x\,x))\,(\lambda z.(\lambda x.x)\,\lambda y.z)$?

β-réduction

- Combien de redex dans $(\lambda x.(\lambda y.y)\,(x\,x))\,(\lambda z.(\lambda x.x)\,\lambda y.z)$? 3
- Réduire $\Omega = (\lambda x.x\,x)\,\lambda y.y\,y$

β-réduction

- Combien de redex dans $(\lambda x.(\lambda y.y)\,(x\,x))\,(\lambda z.(\lambda x.x)\,\lambda y.z)$? 3
- Réduire $\Omega = (\lambda x.x\,x)\,\lambda y.y\,y$
- Forme normale : terme sans redex

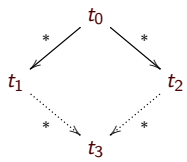
$$x \qquad \lambda x.x \qquad x\,\lambda y.y \qquad \lambda x.x\,\lambda y.y \ldots$$

- Unicité de la forme normale (si elle existe) ?

Confluence

Théorème

Le λ-calcul est confluent.



Corollaire (Unicité de la forme normale)

Si $t \to_\beta^* u_1$ et $t \to_\beta^* u_2$ et u_1 et u_2 sont des formes normales, alors $u_1 = u_2$ (modulo α-conversion).

Sémantique Opérationnelle Structurelle (SOS)

Plusieurs manières de définir la réduction

- Jusqu'ici, réécriture sur les termes
- Sémantique opérationnelle structurelle (*Structural Operational Semantics*, SOS) $t \to t'$
  - opérationnelle : décrit comment le résultat est calculé (vs. dénotationnelle, axiomatique)
  - structurelle : définition inductive sur la structure du terme
  - à petit pas
- D'autres formats de sémantiques possibles (à grand pas, contextuelle, ...)

Sémantique Opérationnelle Structurelle (SOS)

$$\beta\ \frac{}{(\lambda x.t)\,u \to t\{u/x\}} \qquad \text{appL}\ \frac{t \to t'}{t\,u \to t'\,u} \qquad \text{appR}\ \frac{u \to u'}{t\,u \to t\,u'} \qquad \lambda\ \frac{t \to t'}{\lambda x.t \to \lambda x.t'}$$

Exemple : $\mathrm{id}\,(\mathrm{id}\,\lambda z.\mathrm{id}\,z)$

Stratégie de réduction

- Calcul confluent : peu importe le choix du redex
- En pratique, stratégie de réduction
- Pas de réduction dans les λ-abstractions (stratégies *faibles*)
- Appel par nom (*call-by-name*, CBN) : Haskell (preque : *call-by-need*)
- Appel par valeur (*call-by-value*, CBV) : tous les langages sauf Haskell

Appel par nom (CBN)

$$\frac{}{(\lambda x.t)\, u \to t\{u/x\}}\ \beta \qquad \frac{t \to t'}{t\, u \to t'\, u}\ \text{appG}$$

- Évaluer le terme de gauche uniquement
- Faire la β-reduction

Exemple : $\mathrm{id}\,(\mathrm{id}\,\lambda z.\mathrm{id}\,z)$

Appel par valeur (CBV)

$$\begin{array}{lll} \text{valeur} \quad v ::= & x & \text{(variable)} \\ & \lambda x.t & \text{(abstraction)} \end{array}$$

$$\frac{}{(\lambda x.t)\, v \to t\{v/x\}}\ \beta \qquad \frac{u \to u'}{t\, u \to t\, u'}\ \text{appD} \qquad \frac{t \to t'}{t\, v \to t'\, v}\ \text{appG}$$

- Évaluer u puis t : évaluation de droite à gauche
- En pratique : λ-termes de droite à gauche, expressions arithmétiques et booléennes de gauche à droite

Exemple : $\mathrm{id}\,(\mathrm{id}\,\lambda z.\mathrm{id}\,z)$

Appel par valeur vs appel par nom

Rappel : $\Omega = (\lambda x.x\,x)\,(\lambda x.x\,x)$, $\mathrm{id} = \lambda x.x$

- Évaluer $(\lambda xy.y)\,\Omega\,\mathrm{id}$ en appel par valeur et par nom
- Évaluer $\lambda x.(x\,x)(\mathrm{id}\,\mathrm{id})$ en appel par valeur et par nom

	CBV	CBN
duplication du calcul	☺	☹
arguments inutiles, divergence	☹	☺
effets de bords (affichages à l'écran ...)	☺	☹

Appel par valeur vs appel par nom

Rappel : $\Omega = (\lambda x.x\,x)\,(\lambda x.x\,x)$, $\mathrm{id} = \lambda x.x$

- Évaluer $(\lambda xy.y)\,\Omega\,\mathrm{id}$ en appel par valeur et par nom
- Évaluer $\lambda x.(x\,x)(\mathrm{id}\,\mathrm{id})$ en appel par valeur et par nom

	CBV	CBN
duplication du calcul	☺	☹
arguments inutiles, divergence	☹	☺
effets de bords (affichages à l'écran ...)	☺	☹

Appel par nécessité (*call-by-need*, Haskell)

- $(\lambda x.t)\,u$: stocker $x \mapsto u$, évaluer t
- si on a besoin de x, évaluer t en v, mettra à jour $x \mapsto v$

Preuve par induction

Lemme (lemme de substitution)

Pour tout t, u, s, y *et* x, *si* $x \neq y$ *et* $x \notin \mathrm{fv}(s)$, *alors* $t\{u/x\}\{s/y\} = t\{s/y\}\{u\{s/y\}/x\}$.

Démonstration.

Induction sur t. Admis. □

Lemme

Pour tout t, u, s *et* x, $t \to u$ *implique* $t\{s/x\} \to u\{s/x\}$.